

Régularisation et Hyper-paramètres des Réseaux de Neurones

Compte Rendu

Encadree par : Pr. Mohamed CHERRADI

Realise par : Nourhane Elhouari

Filiere : TDIA2

Module : Deep Learning

1. Introduction

L'entraînement d'un réseau de neurones ne se limite pas à définir une architecture et à lancer l'optimisation. La performance finale dépend fortement d'un ensemble de **hyper-paramètres** des paramètres fixés avant l'entraînement et qui ne sont pas appris directement par la descente de gradient.

Ces hyper-paramètres se divisent en deux grandes familles :

- **Hyper-paramètres d'optimisation** : ils contrôlent le processus d'apprentissage (comment le réseau apprend).
- **Hyper-paramètres d'architecture** : ils définissent la structure du réseau (ce que le réseau est capable de représenter).

La régularisation est un mécanisme transversal qui vise à **réduire le sur-apprentissage (overfitting)**, c'est-à-dire le phénomène par lequel le modèle apprend trop précisément les données d'entraînement au détriment de sa capacité à généraliser sur de nouvelles données.

2. Hyper-paramètres d'Optimisation

Les hyper-paramètres d'optimisation agissent directement sur la façon dont l'algorithme d'apprentissage met à jour les poids du réseau lors de la rétropropagation du gradient.

2.1 Pas d'apprentissage (Learning Rate)

Le pas d'apprentissage η est l'un des hyper-paramètres les plus critiques. Il détermine la taille du pas effectué dans la direction opposée au gradient lors de la mise à jour des poids :

$$\theta \leftarrow \theta - \eta \cdot \nabla L(\theta)$$

Un learning rate trop grand provoque des oscillations et empêche la convergence. Un learning rate trop petit ralentit considérablement l'entraînement et risque de bloquer dans un minimum local.

Valeur η	Convergence	Risque
$\eta < 1e-4$	Très lente	Minima locaux
$\eta = 1e-3$	Bonne	Faible
$\eta = 1e-2$	Rapide	Oscillations
$\eta > 0.1$	Instable	Divergence

2.2 Méthodes d'optimisation

Plusieurs algorithmes d'optimisation ont été développés pour améliorer la descente de gradient stochastique (SGD) standard :

SGD - Stochastic Gradient Descent

Mise à jour classique du gradient. Peut être amélioré avec un **momentum** (accumulation d'une fraction du gradient précédent) pour accélérer la convergence et réduire les oscillations.

Adam - Adaptive Moment Estimation

Combine le momentum et une adaptation individuelle du pas d'apprentissage pour chaque paramètre. Calcule la moyenne mobile du gradient (**m**) et de son carré (**v**) robuste au choix de η .

$$m_t = \beta_1 \cdot m_{t-1} + (1-\beta_1) \cdot g_t \qquad v_t = \beta_2 \cdot v_{t-1} + (1-\beta_2) \cdot g_t^2$$

$$\theta_t = \theta_{t-1} - (\eta / \sqrt{v_t + \epsilon}) \cdot m_t \quad (\beta_1=0.9, \beta_2=0.999)$$

RMSprop

Normalise le gradient par une moyenne mobile de sa variance. Adapté aux problèmes avec gradients très hétérogènes (RNN, par exemple).

2.3 Batch Size

Le **batch size** est le nombre d'exemples utilisés pour calculer chaque mise à jour du gradient. Il influence directement la **stabilité du gradient** et le **temps de calcul** par époque.

- **Petit batch (8-64)** : gradient bruité mais régularisant, bonne généralisation, temps plus long.
- **Grand batch (512-4096)** : gradient stable, convergence rapide, mais risque de sur-apprentissage et perte de généralisation.
- **Mini-batch (64-256)** : compromis idéal en pratique.

2.4 Nombre d'Époques

Une **époque** correspond à un passage complet sur l'ensemble des données d'entraînement. Un nombre d'époques insuffisant mène au **sous-apprentissage**, un nombre excessif provoque l'**overfitting**.

La technique d'**early stopping** interrompt l'entraînement lorsque la perte de validation cesse de diminuer, définissant ainsi automatiquement le nombre optimal d'époques.

3. Régularisation (Paramètre λ)

La régularisation ajoute un **terme de pénalité** à la fonction de perte pour contraindre les poids du modèle et réduire l'overfitting. Le paramètre λ contrôle l'intensité de cette régularisation :

$$L\text{-totale} = L\text{-données} + \lambda \cdot \Omega(\theta)$$

3.1 Régularisation L1 (Lasso)

La pénalité L1 est la somme des valeurs absolues des poids : $\Omega(\theta) = \sum |w_i|$

Effet principal : encourage la **parcimonie** des poids : de nombreux poids sont poussés exactement à zéro, réalisant ainsi une sélection automatique de caractéristiques.

Cas d'usage : utile quand on suspecte que seul un sous-ensemble de features est pertinent.

3.2 Régularisation L2 (Ridge / Weight Decay)

La pénalité L2 est la somme des carrés des poids : $\Omega(\theta) = \sum w_i^2$

Effet principal : pousse l'ensemble des poids vers des valeurs **petites mais non nulles**. Pénalise les grands poids sans éliminer complètement les connexions.

Cas d'usage : régularisation standard dans la plupart des problèmes de deep learning. Implémenté comme **weight_decay** dans Adam/SGD.

Critère	L1 (Lasso)	L2 (Ridge)
Pénalité	$\sum w_i $	$\sum w_i^2$
Effet sur les poids	Poids $\rightarrow 0$ (parcimonie)	Poids petits mais $\neq 0$
Sélection de features	Oui (implicite)	Non
Convexité	Oui (non différentiable en 0)	Oui (lisse)
Implémentation PyTorch	Manuelle (boucle)	weightdecay= λ
Valeur λ typique	1e-5 à 1e-3	1e-4 à 1e-2

4. Hyper-paramètres d'Architecture

Les hyper-paramètres d'architecture définissent la capacité représentationnelle du modèle, c'est-à-dire la complexité des fonctions qu'il peut apprendre.

4.1 Nombre de Couches

La profondeur du réseau est fondamentale. Un réseau plus profond peut approximer des fonctions plus complexes grâce à la composition non linéaire des représentations intermédiaires.

- **Réseau peu profond (1-2 couches)** : rapide, moins de risque d'overfitting, limité pour les tâches complexes.
- **Réseau profond(5+ couches)** : très expressif, nécessite davantage de données et de régularisation, risque de vanishing gradient.

4.2 Nombre de Neurones par Couche

La largeur de chaque couche détermine la dimensionnalité des représentations intermédiaires. Une architecture en entonnoir (diminution progressive) est souvent employée pour extraire des représentations de plus en plus abstraites.

4.3 Types de Couches

Type	Abréviation	Description / Usage typique
Fully Connected	FC / Dense	Couche de base, connexion totale entre neurones
Convolution	CNN	Images et données spatiales, partage de poids
Récurrente	RNN	Séquences temporelles, mémoire à court terme
LSTM	LSTM	Séquences longues, gestion de la dépendance à long terme
GRU	GRU	Variante allégée de LSTM, plus rapide à entraîner
Normalisation	BatchNorm	Stabilise l'entraînement, accélère la convergence

4.4 Fonctions d'Activation

La fonction d'activation introduit la non-linéarité qui permet au réseau d'approximer des fonctions complexes.

- **ReLU** : $f(x) = \max(0, x)$. Standard, évite le vanishing gradient, simple à calculer.
- **Tanh** : $f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$. Sortie $\in [-1, 1]$, utile pour les RNN.
- **Sigmoid** : $f(x) = 1/(1+e^{-x})$. Sortie $\in [0,1]$, surtout en sortie pour classification binaire.
- **Leaky ReLU / GELU / Swish** : variantes modernes qui corrigent le problème des neurones "morts" de ReLU.

4.5 Dropout

Le dropout est une technique de régularisation d'architecture : à chaque étape d'entraînement, chaque neurone est **désactivé aléatoirement** avec une probabilité p (taux de dropout). Cela force le réseau à apprendre des représentations redondantes et robustes.

Valeurs typiques : $p \in [0.2, 0.5]$. Le dropout est **désactivé en inférence** (mode eval() en PyTorch).

4.6 Hyper-paramètres spécifiques aux CNN

Hyper-paramètre	Valeurs courantes	Rôle
Taille du filtre	3×3, 5×5, 7×7	Champ réceptif de chaque neurone de convolution
Stride	1, 2	Pas de déplacement du filtre (2 = sous-échantillonnage)
Padding	'same', 'valid'	Préservation ou non des dimensions spatiales
Pooling	MaxPooling, AvgPooling	Réduction de dimension, invariance locale
Nb de filtres	32, 64, 128...	Nombre de features extraites par couche

4.7 Transfer Learning et Architectures Spécifiques

Le transfer learning consiste à réutiliser un modèle pré-entraîné sur une large base de données (ex. ImageNet) et à l'adapter à une tâche spécifique. Deux stratégies principales :

- **Feature extraction** : les couches convolutives sont gelées ; seule la tête de classification est ré-entraînée.
- **Fine-tuning** : l'ensemble du réseau est ré-entraîné avec un learning rate très faible.

Architectures de référence :

- **ResNet** : connexions résiduelles (skip connections) permettant d'entraîner des réseaux très profonds (50, 101, 152 couches).
- **VGG** : architecture simple et profonde, filtres uniformes 3×3. Bonne baseline.

EfficientNet / Vision Transformer (ViT) : architectures modernes alliant performance et efficacité.

5. Partie Implémentation

L'implémentation complète est fournie dans le fichier [hyperparametres.py](#). Ce fichier PyTorch entraîne un réseau de neurones sur le dataset MNIST et réalise 6 expériences illustrant l'impact de chaque famille d'hyper-paramètres.

5.1 Structure du Code

Section	Description
Section 1 - Données	Chargement MNIST, normalisation, DataLoaders
Section 2 -Modèle	Classe ReseauFC : architecture FC configurable
Section 3 - Entraînement	Fonction entrainer() avec support L1/L2
Expérience A	Impact du learning rate (1e-4 à 1e-1)
Expérience B	SGD vs SGD+momentum vs Adam vs RMSprop
Expérience C	Régularisation L1 vs L2 (λ variable)
Expérience D	Taux de dropout (0 à 0.6)
Expérience E	Batch size (16 à 1024)
Expérience F	Profondeur et largeur du réseau

5.2 Comment Exécuter

[pip install torch torchvision matplotlib numpy](#)

[python hyperparametres.py](#)

5.3 Résultats Attendus

L'exécution génère automatiquement 5 graphiques PNG illustrant :

- `resultats_learning_rate.png` - Courbes de perte et précision selon le learning rate
- `resultats_optimiseurs.png` - Comparaison des méthodes d'optimisation
- `resultats_regularisation.png` - Impact de L1 et L2 sur la perte d'entraînement
- `resultats_dropout.png` - Effet du dropout sur le gap train/val
- `resultats_batch_size.png` - Compromis batch size / convergence
- `resultats_architecture.png` - Profondeur vs largeur en termes de précision

6. Synthèse et Bonnes Pratiques

Le tableau ci-dessous récapitule les recommandations pratiques pour le réglage des hyper-paramètres d'un réseau de neurones en deep learning :

Hyper-paramètre	Valeur de départ	Stratégie de réglage
Learning rate	1e-3 (Adam)	Grid search ou scheduler adaptatif
Optimiseur	Adam	Comparer SGD+momentum pour fine- tuning
Batch size	64 ou 128	Ajuster selon GPU disponible
Époques	30-100	Early stopping sur val_loss
Régularisation L2	1e-4	Augmenter si overfitting
Régularisation L1	1e-5	Uniquement si sélection de features
Dropout	0.3-0.5	Après les couches FC uniquement
Nb couches	2-4 (FC)	Augmenter progressivement
Nb neurones	128-512	Décroissant (entonnoir)
Activation	ReLU	Leaky ReLU si neurones morts

Conclusion : les expériences confirment que le learning rate et le choix de l'optimiseur ont l'impact le plus immédiat sur la convergence. La régularisation (L2 + dropout) est essentielle pour réduire le gap train/validation. L'architecture influe principalement sur la capacité du modèle, mais au-delà d'un seuil ($\approx 256k$ paramètres sur MNIST), le gain devient marginal.



